MIPS Architecture Design

Course Name: T225-EGC 121-SecA, Computer Architecture

Anamitra Basu (BC2025008)
Aaronya Chakraborty (IC2025003)
Aatraya Mukherjee (IC2025004)

Introduction

MIPS (**Microprocessor without Interlocked Pipelined Stages)** is a family of reduced instruction set computers (RISC) Instruction Set Architectures (ISA) developed by MIPS Computer Systems, now MIPS Technologies, based in the U.S.

There are multiple versions of MIPS, including MIPS I, II, III, IV, and V, as well as five releases of MIPS32/64 (for 32- and 64-bit implementations, respectively). The early MIPS architectures were 32-bit; 64-bit versions were developed later.

Computer Architecture courses in universities and technical schools often study the MIPS architecture.The architecture greatly influenced later RISC architectures such as Alpha. In March 2021, MIPS announced that the development of the MIPS architecture had ended as the company is making the transition to RISC-V.

Processor

We have implemented three programs in MIPS assembly, namely : Selection Sort, Bubble Sort and Binary Search. We have coded the processor in Python, implementing 16 MIPS commands/instructions, including syscalls.

We have used the Mars4_5 compiler to compile the MIPS assembly code and have stored the binary text and data in separate text files. Our processor reads these files during execution. We have assumed the base address of the instructions as 0x00400000 and data as 0x10010000 respectively.

We have divided the processor into mainly three components or classes namely : i) Register (ii) Memory and (iii)CPU. The CPU class contains the functions to trace the the 5 stages of MIPS architecture for any instruction namely: IF, ID, EX, MEM, WB. The file containing the processor is named Processor.py.

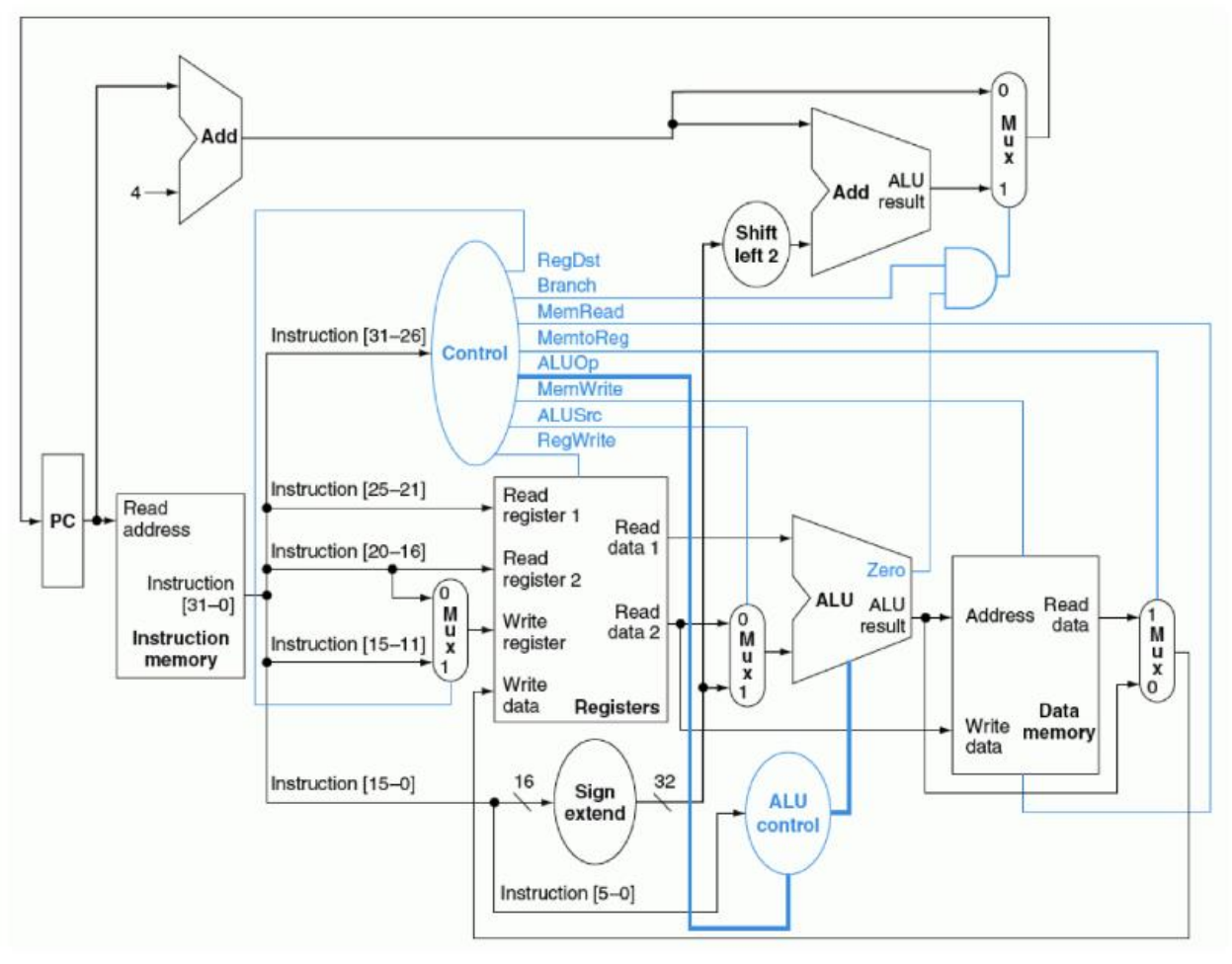

Figure : MIPS Architecture

Selection Sort

This program takes a sequence of n integers and sorts the array in ascending order.

Best case time complexity- O(n^2)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n = 5;
    int arr[5] = {7, 3, 9, 1, 5};

    for(int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for(int j = i + 1; j < n; j++) {
            if(arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }

    return 0;
}
```

Binary Search

This program takes a sorted array and a target element. It searches for the target in the array and returns the index (0 indexed) if found, otherwise returns –1.

```cpp
#include<bits/stdc++.h>
using namespace std;

int main()
{
    vector<int> a={3, 4, 7, 78 , 100, 121, 256};
    int target=4,ans=-1;
    int r=6,l=0;

    while(l<=r)
    {
        int mid = (l+r)/2;

        if(a[mid]==target)
        {
            ans=mid;
            break;
        }
        else if(a[mid]<target)
        {
            l=mid+1;
        }
        else
        {
            r=mid-1;
        }
    }

    //ans contains the answer
}
```

Bubble Sort

This program takes a sequence of n integers and sorts the array in ascending order.

Best case time complexity – O(n)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n = 5;
    int arr[5] = {8, 2, 9, 1, 9};

    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    return 0;
}
```

Definitions of Components used

1. OPCODE DICTIONARY

The Opcode Dictionary is responsible for instruction decoding within the processor. It maps each MIPS instruction mnemonic to its corresponding opcode, function field (for R-type instructions), and associated control signals. In this implementation, two separate dictionaries are maintained: one for R-type instructions and another for I-type and J-type instructions. During the Instruction Decode (ID) stage, the opcode and function bits are extracted from the 32-bit instruction and matched with entries in these dictionaries. Once a match is found, the corresponding control signals are activated. This mechanism enables structured and accurate instruction execution.

2. REGISTER CLASS

The Register class simulates the MIPS register file and maintains the processor state. It includes storage for general-purpose registers such as $t0–$t9, $s0–$s7, $a0–$a3, $v0–$v1, $at, and $ra. The program counter (PC) is also maintained within this class and is initialized to the base address of the text segment. The $zero register is hardwired to always return zero. Register values are read during the Execute stage and updated during the Write Back stage. This class ensures proper storage and retrieval of intermediate and final computational results.

3. MEMORY CLASS

The Memory class simulates both instruction memory and data memory using external files. Instruction memory stores 32-bit machine code instructions, while data memory stores 32-bit data words. Address translation is performed relative to predefined base addresses for text and data segments. The class provides methods to read and write instructions and data in binary format. All values are stored as 32-bit representations to maintain architectural consistency. This implementation follows a Harvard-style separation of instruction and data memory.

4. ARITHMETIC LOGIC UNIT (ALU)

The Arithmetic Logic Unit (ALU) performs all arithmetic and logical operations during the Execute (EX) stage of the processor. It receives operands either from registers or immediate fields depending on control signals. The ALU supports operations such as addition, subtraction, logical AND, logical OR, shift operations, and set-less-than comparisons. It also calculates effective addresses for load and store instructions. For branch instructions, it performs comparisons to determine whether a branch condition is satisfied. The ALU output is forwarded to either the memory stage or the write-back stage as required.

5. CONTROL DICTIONARY

The Control Dictionary defines the control signal configuration for each supported instruction. It specifies signals such as RegDst, ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch, Jump, and ALUOp. These signals determine how data flows through the datapath during execution. During the Instruction Decode stage, the appropriate set of control signals is selected based on the instruction opcode. This structured approach simplifies control logic implementation. The Control Dictionary effectively represents the processor's control unit behavior.

6. CONTROL LINES

Control lines are binary signals generated during the Decode stage to manage processor operations. These signals control multiplexers, memory access, register writing, and program counter updates. For example, RegWrite enables register updates, while MemRead and MemWrite manage data memory access. The Branch and Jump signals modify control flow by updating the program counter when necessary. ALUSrc and RegDst determine operand selection and destination register selection. Together, these control lines coordinate the functioning of all five stages: IF, ID, EX, MEM, and WB.

Processor

Processor code:

```
TEXT_BASE = 0x00400000
DATA_BASE = 0x10010000


R_TYPE_OPCODE = {

    "add":  {"opcode":"000000","funct":"100000",
             "RegDst":1,"ALUSrc":0,"MemtoReg":0,
             "RegWrite":1,"MemRead":0,"MemWrite":0,
             "Branch":0,"Jump":0,"ALUOp":"10"},

    "sub":  {"opcode":"000000","funct":"100010",
             "RegDst":1,"ALUSrc":0,"MemtoReg":0,
             "RegWrite":1,"MemRead":0,"MemWrite":0,
             "Branch":0,"Jump":0,"ALUOp":"10"},

    "and":  {"opcode":"000000","funct":"100100",
             "RegDst":1,"ALUSrc":0,"MemtoReg":0,
             "RegWrite":1,"MemRead":0,"MemWrite":0,
             "Branch":0,"Jump":0,"ALUOp":"10"},

    "or":   {"opcode":"000000","funct":"100101",
             "RegDst":1,"ALUSrc":0,"MemtoReg":0,
             "RegWrite":1,"MemRead":0,"MemWrite":0,
             "Branch":0,"Jump":0,"ALUOp":"10"},

    "slt":  {"opcode":"000000","funct":"101010",
             "RegDst":1,"ALUSrc":0,"MemtoReg":0,
             "RegWrite":1,"MemRead":0,"MemWrite":0,
             "Branch":0,"Jump":0,"ALUOp":"10"},

    "sll":  {"opcode":"000000","funct":"000000",
             "RegDst":1,"ALUSrc":0,"MemtoReg":0,
             "RegWrite":1,"MemRead":0,"MemWrite":0,
             "Branch":0,"Jump":0,"ALUOp":"10"},

    "srl":  {"opcode":"000000","funct":"000010",
             "RegDst":1,"ALUSrc":0,"MemtoReg":0,
             "RegWrite":1,"MemRead":0,"MemWrite":0,
             "Branch":0,"Jump":0,"ALUOp":"10"},

    "syscall": {"opcode":"000000","funct":"001100",
            "RegDst":None,"ALUSrc":None,"MemtoReg":None,
            "RegWrite":0,"MemRead":0,"MemWrite":0,
            "Branch":0,"Jump":0,"ALUOp":None}

}


OPCODE_FUNCT = {
    "addi": {"opcode":"001000","funct":None,
             "RegDst":0,"ALUSrc":1,"MemtoReg":0,
             "RegWrite":1,"MemRead":0,"MemWrite":0,
             "Branch":0,"Jump":0,"ALUOp":"00"},
```

```
    "ori":  {"opcode":"001101","funct":None,
             "RegDst":0,"ALUSrc":1,"MemtoReg":0,
             "RegWrite":1,"MemRead":0,"MemWrite":0,
             "Branch":0,"Jump":0,"ALUOp":"11"},

    "lui": {"opcode":"001111","funct":None,
        "RegDst":0,"ALUSrc":1,"MemtoReg":0,
        "RegWrite":1,"MemRead":0,"MemWrite":0,
        "Branch":0,"Jump":0,"ALUOp":"LUI"},


    "lw":   {"opcode":"100011","funct":None,
             "RegDst":0,"ALUSrc":1,"MemtoReg":1,
             "RegWrite":1,"MemRead":1,"MemWrite":0,
             "Branch":0,"Jump":0,"ALUOp":"00"},

    "sw":   {"opcode":"101011","funct":None,
             "RegDst":None,"ALUSrc":1,"MemtoReg":None,
             "RegWrite":0,"MemRead":0,"MemWrite":1,
             "Branch":0,"Jump":0,"ALUOp":"00"},


    "beq":  {"opcode":"000100","funct":None,
             "RegDst":None,"ALUSrc":0,"MemtoReg":None,
             "RegWrite":0,"MemRead":0,"MemWrite":0,
             "Branch":1,"Jump":0,"ALUOp":"01"},

    "bne":  {"opcode":"000101","funct":None,
             "RegDst":None,"ALUSrc":0,"MemtoReg":None,
             "RegWrite":0,"MemRead":0,"MemWrite":0,
             "Branch":1,"Jump":0,"ALUOp":"01"},


    "j":    {"opcode":"000010","funct":None,
             "RegDst":None,"ALUSrc":None,"MemtoReg":None,
             "RegWrite":0,"MemRead":0,"MemWrite":0,
             "Branch":0,"Jump":1,"ALUOp":None}
}




class Register:
    def __init__(self):
        self.t = [0] * 10
        self.s = [0] * 8
        self.a = [0] * 4

        self.pc = TEXT_BASE

        self.zero = 0
```

```python
        self.zero = 0
        self.at = 0
        self.ra = 0
        self.v0 = 0
        self.v1 = 0



class Memory:

    def __init__(self, file_instr,file_data):
        self.data_memory = file_data
        self.instr_memory = file_instr


    def read_data(self, addr):
        with open(self.data_memory, "r") as f:
            for i, line in enumerate(f):
                if i == addr:
                    return line.strip()
        return ""


    def read_instr(self, addr):
        with open(self.instr_memory, "r") as f:
            for i, line in enumerate(f):
                if i == addr:
                    return line.strip()
        return ""

    def write_data(self, addr, value):
        value &= ((1 << 32) - 1)

        with open(self.data_memory, "r") as f:
            lines = f.readlines()

        while len(lines) <= addr:
            lines.append("0".zfill(32) + "\n")

        lines[addr] = format(value, "032b") + "\n"

        with open(self.data_memory, "w") as f:
            f.writelines(lines)


    def write_instr(self, addr, value):
        value &= ((1 << 32) - 1)

        with open(self.instr_memory, "r") as f:
            lines = f.readlines()

        while len(lines) <= addr:
            lines.append("0".zfill(32) + "\n")
```

```python
        while len(lines) <= addr:

        lines[addr] = format(value, "032b") + "\n"

        with open(self.instr_memory, "w") as f:
            f.writelines(lines)



class CPU:
    def __init__(self,instr_file,data_file):
        self.mem = Memory(instr_file,data_file)
        self.reg = Register()
        self.running = True
        self.instr=""
        self.data=""
        self.opcode =""
        self.rs=""
        self.rt=""
        self.rd=""
        self.shamt=""
        self.func = ""
        self.imme = 0
        self.jaddr = 0
        self.ctrl = None
        self.current_instr = ""
        self.alu_result = 0
        self.mem_data = 0
        self.cycle=0


    def print_reg(self):
        print("\n============================")
        print(f"Cycle: {self.cycle}")
        print(f"PC: {hex(self.reg.pc)}")

        print("Registers:")

        print("---- $a registers ----")
        for i in range(4):
            if self.reg.a[i]!=0:
                print(f"$a{i} = {self.reg.a[i]}")

        print("---- $t registers ----")
        for i in range(10):
            if self.reg.t[i]!=0:
                print(f"$t{i} = {self.reg.t[i]}")

        print("---- $s registers ----")
        for i in range(8):
            if self.reg.s[i]!=0:
                print(f"$s{i} = {self.reg.s[i]}")

        print("============================\n")
```

```python

    def IF_stage(self):
        self.instr = self.mem.read_instr((self.reg.pc-TEXT_BASE) // 4)
        if not self.instr: # Stop if no more instructions
            self.running = False
            return
        self.reg.pc += 4


    def ID_stage(self):
        self.ctrl = None
        self.current_instr = None

        self.opcode = self.instr[0:6]
        self.rs = int(self.instr[6:11],2)
        self.rt = int(self.instr[11:16],2)

        if self.opcode == "000000":
            self.rd = int(self.instr[16:21],2)
            self.shamt = int(self.instr[21:26],2)
            self.func = self.instr[26:32]


            for name,signals in R_TYPE_OPCODE.items():
                if signals["funct"] == self.func:
                    self.ctrl = signals.copy()
                    self.current_instr = name
                    break


        elif self.opcode in ["000010"] :
            self.jaddr = int(self.instr[6:32],2)
            self.current_instr = "j"

            self.ctrl = {"opcode":"000010","funct":None,
             "RegDst":None,"ALUSrc":None,"MemtoReg":None,
             "RegWrite":0,"MemRead":0,"MemWrite":0,
             "Branch":0,"Jump":1,"ALUOp":None}

        else:
            self.rd = None
            raw_imm = int(self.instr[16:32], 2)
            if raw_imm & 0x8000:
                self.imme = raw_imm - 0x10000
            else:
                self.imme = raw_imm

            for name, signals in OPCODE_FUNCT.items():
                if signals["opcode"] == self.opcode:
                    self.ctrl = signals.copy()
                    self.current_instr = name
                    break

```

```python
        if self.ctrl is None:
            print("ERROR: Unknown instruction")
            print("Opcode:", self.opcode)
            self.running = False


    def get_register_val(self, register_idx):
        if register_idx == 0: return 0  # $zero
        if register_idx == 1: return self.reg.at
        if register_idx == 2: return self.reg.v0
        if register_idx == 3: return self.reg.v1
        if 4 <= register_idx <= 7: return self.reg.a[register_idx - 4]
        if 8 <= register_idx <= 15: return self.reg.t[register_idx - 8]
        if 16 <= register_idx <= 23: return self.reg.s[register_idx - 16]
        if 24 <= register_idx <= 25: return self.reg.t[register_idx - 16] # t8-t9
        if register_idx == 31: return self.reg.ra
        return 0

    def EX_stage(self):


        # 1. Fetch Operands
        val_at_rs = self.get_register_val(self.rs)
        val_at_rt = self.get_register_val(self.rt)

        # 2. ALU Mux: Choose between Register (0) or Immediate (1)
        operand_b = val_at_rt  # default

        if self.ctrl and self.ctrl["ALUSrc"] is not None:
            operand_b = val_at_rt if self.ctrl["ALUSrc"] == 0 else self.imme


        # 3. ALU Operations
        if self.current_instr == "add":
            self.alu_result = val_at_rs + operand_b

        elif self.current_instr == "sub":
            self.alu_result = val_at_rs - operand_b

        elif self.current_instr == "and":
            self.alu_result = val_at_rs & operand_b

        elif self.current_instr == "or":
            self.alu_result = val_at_rs | operand_b

        elif self.current_instr == "sll":
            # Shift rt, not rs
            self.alu_result = val_at_rt << self.shamt
```

```python

        elif self.current_instr == "srl":
            # Shift rt, not rs
            self.alu_result = val_at_rt >> self.shamt

        elif self.current_instr == "slt":
            self.alu_result = 1 if val_at_rs < operand_b else 0

        elif self.current_instr == "addi":
            self.alu_result = val_at_rs + operand_b

        elif self.current_instr == "ori":
            self.alu_result = val_at_rs | operand_b

        elif self.current_instr == "lui":
            self.alu_result = self.imme << 16

        # 4. Memory Address Calculation (Base + Offset)
        elif self.current_instr in ["lw", "sw"]:
            self.alu_result = val_at_rs + self.imme

        # 5. Branch Operations
        # Target = PC + (Offset * 4). Note: PC was already +4 in IF_stage.
        elif self.current_instr == "beq":
            if val_at_rs == val_at_rt:
                self.reg.pc += (self.imme * 4)

        elif self.current_instr == "bne":
            if val_at_rs != val_at_rt:
                self.reg.pc += (self.imme * 4)


        elif self.current_instr == "j":
            pc_upper = (self.reg.pc & 0xF0000000)
            pc_upper = (self.reg.pc & 0xF0000000)
            self.reg.pc = pc_upper | (self.jaddr * 4)
    # Concatenate PC upper bits with jaddr shifted left by 2

        elif self.current_instr == "syscall":
            if self.reg.v0 == 10:
                self.running = False



    def MEM_stage(self):

        if not self.ctrl:
            return

        if self.ctrl["MemRead"] == 1 or self.ctrl["MemWrite"] == 1:
            print("ALU_RESULT:", hex(self.alu_result))
            address = (self.alu_result - DATA_BASE)//4

```

```python
        if self.ctrl["MemRead"] == 1:
            raw_data = self.mem.read_data(address)
            if raw_data:
                val = int(raw_data, 2)
                # Check if the sign bit (bit 31) is 1
                self.mem_data = val - 0x100000000 if val & 0x80000000 else val
            else:
                self.mem_data = 0

        if self.ctrl["MemWrite"] == 1:
            val_at_rt = self.get_register_val(self.rt)

            self.mem.write_data(address,val_at_rt)


    def WB_stage(self):
        if not self.ctrl:
            return
        if self.ctrl["RegWrite"] == 0:
            return

        write_value = self.mem_data if self.ctrl["MemtoReg"] == 1 else self.alu_result
        dest_reg = self.rd if self.ctrl["RegDst"] == 1 else self.rt

        if dest_reg == 0: return

        # Mapping logic (must match get_register_val)
        if dest_reg == 2: self.reg.v0 = write_value
        elif dest_reg == 1: self.reg.at = write_value
        elif dest_reg == 3: self.reg.v1 = write_value
        elif 4 <= dest_reg <= 7:
            self.reg.a[dest_reg - 4] = write_value
        elif 8 <= dest_reg <= 15: self.reg.t[dest_reg - 8] = write_value
        elif 16 <= dest_reg <= 23: self.reg.s[dest_reg - 16] = write_value
        elif 24 <= dest_reg <= 25: self.reg.t[dest_reg - 16] = write_value
        elif dest_reg == 31: self.reg.ra = write_value


    def run(self):
        while self.running:
            self.cycle +=1


            self.IF_stage()
            if not self.running: break
            self.ID_stage()
            self.EX_stage()
            self.MEM_stage()
            self.WB_stage()

            if self.cycle % 10 == 0:
                self.print_reg()
```

```python
def get_files():
    file_instr = input("Enter instruction filename: ")
    file_data = input("Enter data filename: ")
    return (file_instr,file_data)

file_list=get_files()
file_instr=file_list[0]
file_data=file_list[1]


cpu=CPU(file_instr,file_data)
cpu.run()
```

MIPS Assembly Codes:

1.Selection Sort

```
.data
A:  .word 7, 3, 9, 1, 5
n:  .word 5

.text
.globl main

main:

    lui  $a0, 0x1001
    ori  $a0, $a0, 0x0000

    lui  $t9, 0x1001
    ori  $t9, $t9, 0x0014
    lw   $a1, 0($t9)

    addi $t0, $zero, 0        # i = 0

outer_loop:
    addi $t7, $a1, -1
    slt  $t8, $t0, $t7
    beq  $t8, $zero, exit

    add  $t1, $t0, $zero
    addi $t2, $t0, 1

inner_loop:
    slt  $t8, $t2, $a1
    beq  $t8, $zero, swap

    sll  $t3, $t2, 2
    add  $t3, $t3, $a0
    lw   $t4, 0($t3)

    sll  $t5, $t1, 2
    add  $t5, $t5, $a0
    lw   $t6, 0($t5)

    slt  $t8, $t4, $t6
    beq  $t8, $zero, skip_update
```

```
        add  $t1, $t2, $zero

    skip_update:
        addi $t2, $t2, 1
        j inner_loop

    swap:
        beq  $t1, $t0, next_i

        sll  $t3, $t0, 2
        add  $t3, $t3, $a0
        lw   $t4, 0($t3)

        sll  $t5, $t1, 2
        add  $t5, $t5, $a0
        lw   $t6, 0($t5)

        sw   $t6, 0($t3)
        sw   $t4, 0($t5)

    next_i:
        addi $t0, $t0, 1
        j outer_loop

    exit:
        addi $v0, $zero, 10
        syscall
```

2. Binary Search

```
.data
    a: .word 3, 4, 7, 78, 100, 121, 256
    k_val: .word 4

.text
.globl main
main:
    la   $s0, a         #Get actual base address
    addi $s1, $zero, 0   #l = 0
    addi $s2, $zero, 6  #r = 5 (size - 1)
    lw   $s3, k_val    #Better to load k from memory too

loop:
    #Check if l > r
    slt  $t0, $s2, $s1      #If r < l, $t0 = 1
    bne  $t0, $zero, fail   #If r < l, jump to fail

    # mid = (l + r)/2
    add  $t1, $s1, $s2      #t1 = l + r
    srl  $s4, $t1, 1       #mid = (l + r) / 2

    #Load a[mid] and check if a[mid] < k
    #MIPS requires calculating byte offset (index * 4)
    sll  $t2, $s4, 2        #t2 = mid * 4
    add  $t2, $t2, $s0      #t2 = address of a[mid]
    lw   $s5, 0($t2)        #$s5 = a[mid]

    slt  $t3, $s5, $s3      #If a[mid] < k, $t3 = 1
    bne  $t3, $zero, l_up   #If a[mid] < k, go to l = mid + 1

    #Check if a[mid] == k or a[mid] > k
    beq  $s5, $s3, found    #If a[mid] == k, go to found

    # r = mid - 1 (If a[mid] > k)
    addi $s2, $s4, -1       #r = mid - 1
    j    loop

l_up:
    #l = mid + 1
    addi $s1, $s4, 1        #l = mid + 1
    j    loop
```

```
        j    loop

    found:
        #Store result and stop execution
        add  $s6, $s4, $zero    # Store mid in result register
        j    done

    fail:
        addi $s6, $zero, -1
            # Result = -1 (not found)

    done:
        # tells the simulator to stop
```

3. Bubble Sort

```
.data
A:  .word 8, 2, 9, 1, 9
n:  .word 5

.text
.globl main

main:

    lui  $a0, 0x1001
    ori  $a0, $a0, 0x0000

    lui  $t9, 0x1001
    ori  $t9, $t9, 0x0014
    lw   $a1, 0($t9)

    addi $t0, $zero, 0        #i = 0

outer_loop:
    addi $t7, $a1, -1         #n-1
    slt  $t8, $t0, $t7
    beq  $t8, $zero, exit     #if i >= n-1 exit

    addi $t1, $zero, 0        #j = 0

inner_loop:
    sub  $t6, $a1, $t0        #n - i
    addi $t6, $t6, -1         #n - i - 1

    slt  $t8, $t1, $t6
    beq  $t8, $zero, next_i   #if j >= n-i-1 go next i

    # Load A[j]
    sll  $t2, $t1, 2
    add  $t2, $t2, $a0
    lw   $t3, 0($t2)
```

```
        # Load A[j+1]
        addi $t4, $t1, 1
        sll  $t4, $t4, 2
        add  $t4, $t4, $a0
        lw   $t5, 0($t4)

        # if A[j] > A[j+1]
        slt  $t8, $t5, $t3        #(A[j+1] < A[j])
        beq  $t8, $zero, no_swap

        # swap
        sw   $t5, 0($t2)
        sw   $t3, 0($t4)

    no_swap:
        addi $t1, $t1, 1
        j inner_loop

    next_i:
        addi $t0, $t0, 1
        j outer_loop

    exit:
        addi $v0, $zero, 10
        syscall
```

Execution Results

The below screenshots show the result after running the processor.py , using only the binary files for the instruction and data segment.

Selection Sort

Before Sorting (the data segment):

```
00000000000000000000000000000111
00000000000000000000000000000011
00000000000000000000000000001001
00000000000000000000000000000001
00000000000000000000000000000101
00000000000000000000000000000101
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
```

After Sorting(the data segment):

```
00000000000000000000000000000001
00000000000000000000000000000011
00000000000000000000000000000101
00000000000000000000000000000111
00000000000000000000000000001001
00000000000000000000000000000101
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
```

Figure: Result of Selection Sort.

Binary Search:

We search on the array [3, 4, 7, 78, 100, 121, 256] for the element 4.

$s6 should contain the index of the element.

```
Enter instruction filename: BS_Text.txt
Enter data filename: BS_Data.txt
ALU_RESULT: 0x1001001c

==============================
Cycle: 10
PC: 0x400028
Registers:
---- $a registers ----
---- $t registers ----
$t1 = 6
---- $s registers ----
$s0 = 268500992
$s2 = 6
$s3 = 4
$s4 = 3
==============================

ALU_RESULT: 0x1001000c

==============================
Cycle: 20
PC: 0x400020
Registers:
---- $a registers ----
---- $t registers ----
$t1 = 6
$t2 = 268501004
---- $s registers ----
$s0 = 268500992
$s2 = 2
$s3 = 4
$s4 = 3
$s5 = 78
==============================

ALU_RESULT: 0x10010004

==============================
Cycle: 30
PC: 0x40005c
Registers:
---- $a registers ----
---- $t registers ----
$t1 = 2
$t2 = 268500996
---- $s registers ----
$s0 = 268500992
$s2 = 2
$s3 = 4
$s4 = 1
$s5 = 4
$s6 = 1
==============================
```

Figure: Result of Binary Search.

Bubble Sort:

Before Sorting(the data segment)

```
00000000000000000000000000001000
00000000000000000000000000000010
00000000000000000000000000001001
00000000000000000000000000000001
00000000000000000000000000001001
00000000000000000000000000000101
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
```

After Sorting(the data segment)

```
00000000000000000000000000000001
00000000000000000000000000000010
00000000000000000000000000001000
00000000000000000000000000001001
00000000000000000000000000001001
00000000000000000000000000000101
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
```

Figure: Result of Bubble Sort

References:

- https://en.wikipedia.org/wiki/Stanford_MIPS
- Computer organization and design MIPS Edition, Henessey and Patterson (H&P). 5th edition
- Digital design and computer architecture, Harris and Harris.